SURF Invite – MidPoint Integration

MidPoint-leading reference architecture and operator guide

Reference implementation
(midpoint-invite-integration, branch midpoint-leading)

November 4, 2026

Abstract

This document describes a reference stack that integrates **SURFconext Invite** with **MidPoint**. **MidPoint is the system of record for roles/groups**: an invite_bridge service creates and deletes matching Roles in Invite via the Invite external API, and notifies Invite when a MidPoint administrator unassigns an Invite-provisioned membership (by_provisioned_scim) so VOOT/AA and app access do not drift. **Invite** remains the invitation UX (and may be started from MidPoint via an **Invite user** menu link); after a guest accepts, Invite provisions **users and memberships** into MidPoint over institutional **SCIM 2.0**. An optional OIDC demo application is included for verification; Moodle and OpenLDAP are parked (code retained, not in the default compose).

Contents

1 Goals and design rationale **3**

2 System overview **3**
2.1 Components 3
2.2 Logical architecture 4
2.3 Repository layout 4

3 MidPoint-leading provisioning flow **4**
3.1 SURF Invite provisioner configuration 5
3.2 Detailed steps 6
3.3 Reusable custom code 7
3.4 SCIM ↔ MidPoint mapping 7

4 MidPoint GUI **7**

5 Invite user from MidPoint **8**

6 OIDC application (membership pull) **10**
6.1 Operator flow 10
6.2 Screenshots 11
6.3 Health endpoint 12

7 Deployment and configuration **12**
7.1 Start the stack 12
7.2 Important environment variables 13
7.3 Traefik hostnames 13

8 Operational notes and lessons learned **13**

9 Parked components (not in default compose) **14**

10 Summary **14**

A Quick reference commands **14**

1 Goals and design rationale

The reference answers three practical questions:

1. How can an institution create roles in **MidPoint** and have them appear automatically in **SURF Invite** for invitation workflows?
2. How can a MidPoint administrator **invite a guest** without leaving MidPoint, while still using Invite's invitation email and accept flow?
3. How do invited guests still land in MidPoint as users with role assignments via **SCIM**, without Invite creating duplicate Roles?
4. How does MidPoint notify Invite when an administrator **revokes** an Invite-provisioned membership (drift prevention)?
5. What minimal custom code must another MidPoint deployment reuse (invite_bridge + SCIM MidPoint plugin leading mode)?

Core principle. MidPoint owns the Role lifecycle (create/delete). Invite owns inviting and accepting guests. SCIM carries Users and memberships from Invite into MidPoint. When Invite echoes a SCIM Group create after the bridge created the Invite Role, the SCIM MidPoint plugin (MIDPOINT_LEADING_GROUPS) matches the existing Role by name and does not create a second Role. When a MidPoint admin unassigns an Invite-SCIM user from an Invite-SCIM role, invite_bridge calls Invite DELETE . . . /user_roles/by_provisioned_scim/{userScimId}/{groupScimId}.

2 System overview

2.1 Components

Component	Role	Hostname (Traefik)
MidPoint + Postgres	IDM/IGA; SoR for Roles; users, assignments	midpoint.$DOMAIN
invite_bridge	MidPoint Roles ↔ Invite API; membership revoke notify; **Invite user** form	invite-bridge.$DOMAIN
SURF Invite	Invitation UX; SCIM client for Users/memberships	(external)
SCIM service	Institutional SCIM API → MidPoint REST	scim.$DOMAIN
OIDC Application	Optional membership-pull demo	app.$DOMAIN
SURFconext / eduID	OIDC authentication	(external)

Table 1: Stack components and Traefik hostnames (default compose). Moodle and OpenLDAP are parked—see Section 9.

2.2 Logical architecture

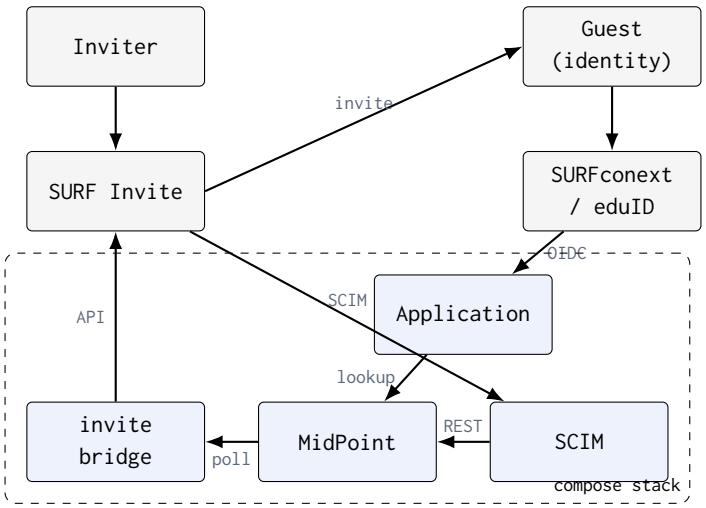


Figure 1: Logical architecture (MidPoint-leading). Roles and membership revokes flow MidPoint → Invite via invite_bridge; users/memberships flow Invite → SCIM → MidPoint. The OIDC app is an optional demo target.

2.3 Repository layout

```
compose.yml / compose.traefik.yml / Makefile
invite_bridge/                # Roles <-> Invite + revoke notify + form
scripts/invite-create-role.sh
scripts/integration_membership_revoke.py
application/                  # OIDC membership-pull demo (optional)
midpoint/post-initial-objects/ # scimResource schema (+ parked LDAP XML)
docs/latex/                   # this operator guide
design.md                     # MidPoint-leading sequence (Mermaid)
```

3 MidPoint-leading provisioning flow

MidPoint owns Role create/delete. Invite owns invitations. Figure 2 shows the sequence (canonical source: repository design.md Mermaid diagram; PNG checked in for PDF builds).

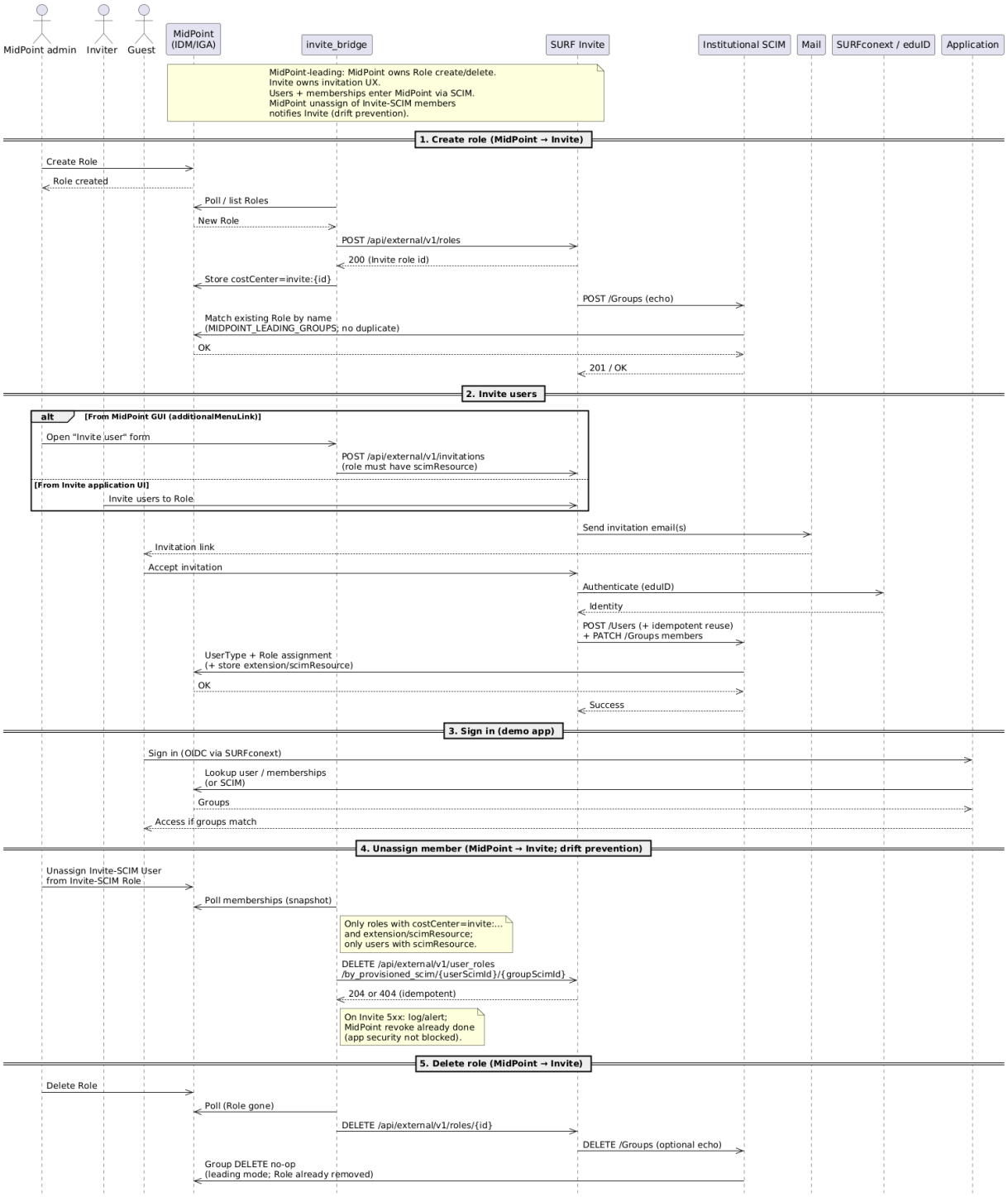


Figure 2: MidPoint-leading sequence: create Role, invite guest, sign-in, unassign member (drift prevention), delete Role.

3.1 SURF Invite provisioner configuration

Before Invite can drive MidPoint, the institutional SCIM endpoint must be registered as a provisioning service in SURF Conext Manage. Figure 3 shows the metadata used for this demo.

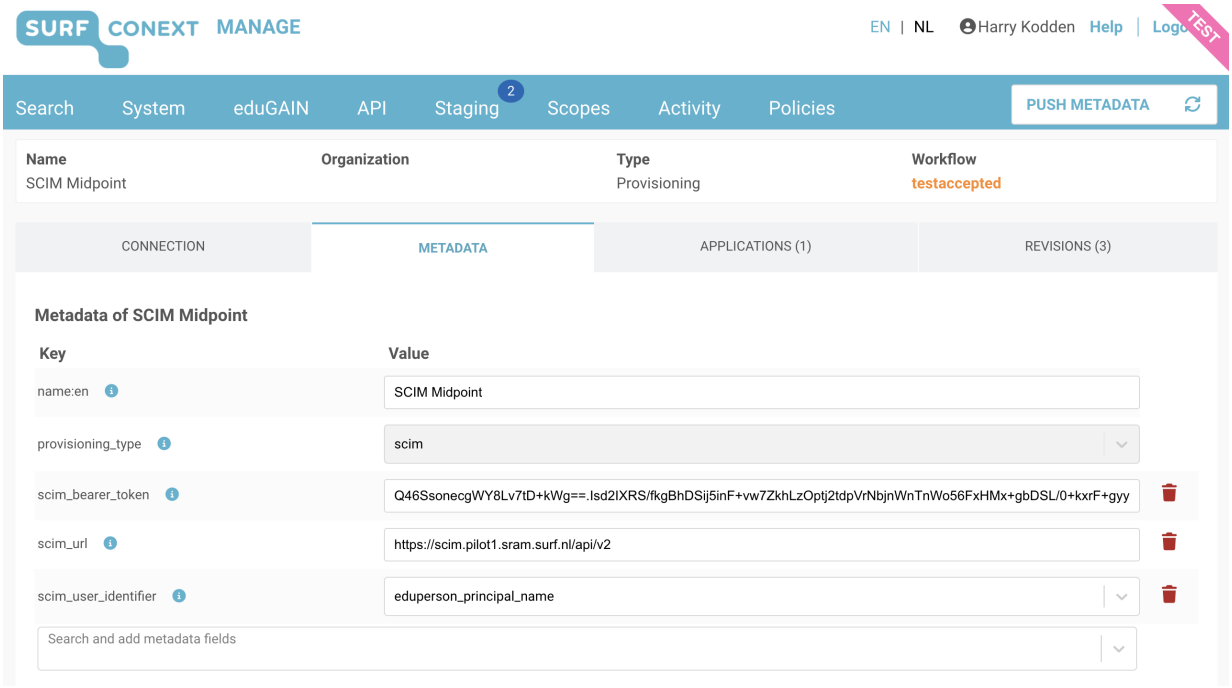


Figure 3: SURF Conext Manage: metadata for the SCIM Midpoint provisioning service (type scim), pointing at the demo SCIM base URL and using eduperson_principal_name as the user identifier.

The essential fields are:

provisioning_type
Set to scim so Invite pushes Users/Groups over SCIM 2.0.

scim_url
Base URL of the institutional SCIM API, including the path prefix (here https://scim.pilot1.sram.surf.nl/ap

scim_bearer_token
Shared secret presented as an HTTP Bearer token; must match SCIM_API_KEY in the compose stack.

scim_user_identifier
Claim used to correlate Invite guests with SCIM/MidPoint users (eduperson_principal_name in this setup—the same key the demo application uses via USER_LOOKUP_CLAIMS).

With this metadata in place (and Invite using remote Manage, not static seed apps), Invite can push Users/memberships (and Group echoes) to SCIM.

3.2 Detailed steps

1. Create role (MidPoint-leading)
MidPoint administrator creates an application Role. invite_bridge polls MidPoint and calls Invite POST /api/external/v1/roles, then stores costCenter=invite:{id} on the MidPoint Role. Invite may echo POST /Groups to SCIM; with MIDPOINT_LEADING_GROUPS the plugin reuses the existing Role by name (no duplicate).

2. Invite and accept
An invitation can be started either in the **Invite application UI** or from MidPoint via the **Invite user** menu (Section 5). After eduID authentication, Invite issues SCIM User create/update and Group membership changes. SCIM creates/updates MidPoint UserType and

role assignments, and stores the raw SCIM document on extension/scimResource. With SCIM_IDEMPOTENT_USER_CREATE, a re-POST for an existing userName/externalId reuses the User (no 409).

3. Application sign-in
Guest authenticates to the demo application via SURFconext/eduID (OIDC). The application resolves the lookup key (e.g. eduperson_principal_name) and evaluates memberships via MidPoint or SCIM.

4. Unassign member (drift prevention)
When a MidPoint admin removes an Invite-SCIM user from an Invite-SCIM role, invite_bridge calls DELETE /api/external/v1/user_roles/by_provisioned_scim/{userScimId}/{groupScimId} using MidPoint's SCIM resource ids. Invite 404 is treated as success; other Invite errors are logged while MidPoint revoke stands.

5. Delete role
Deleting the MidPoint Role causes invite_bridge to DELETE the Invite Role. Invite-initiated membership remove still flows over SCIM into MidPoint.

3.3 Reusable custom code

invite_bridge/
Python service: MidPoint Role create/delete ↔ Invite external API; membership revoke → by_provisioned_scim; Invite-user form. Required for MidPoint-leading.

SCIM MidPoint plugin
Sibling scim checkout; set MIDPOINT_LEADING_GROUPS=true so Invite Group echoes do not create or delete MidPoint Roles incorrectly. Prefer SCIM_IDEMPOTENT_USER_CREATE=true for Invite re-provision.

3.4 SCIM ↔ MidPoint mapping

SCIM	MidPoint	Notes
Users	UserType	userName→name, displayName→fullName
Groups	RoleType	displayName→name/displayName
Group members	User assignment	targetRef to RoleType
SCIM document	extension/scimResource (legacy: description)	Hidden schema-extension attribute; marks Invite-provisioned objects

Table 2: SCIM MidPoint plugin mapping used by this stack.

4 MidPoint GUI

Administrators work primarily in the MidPoint web UI at https://midpoint.$DOMAIN/midpoint/.

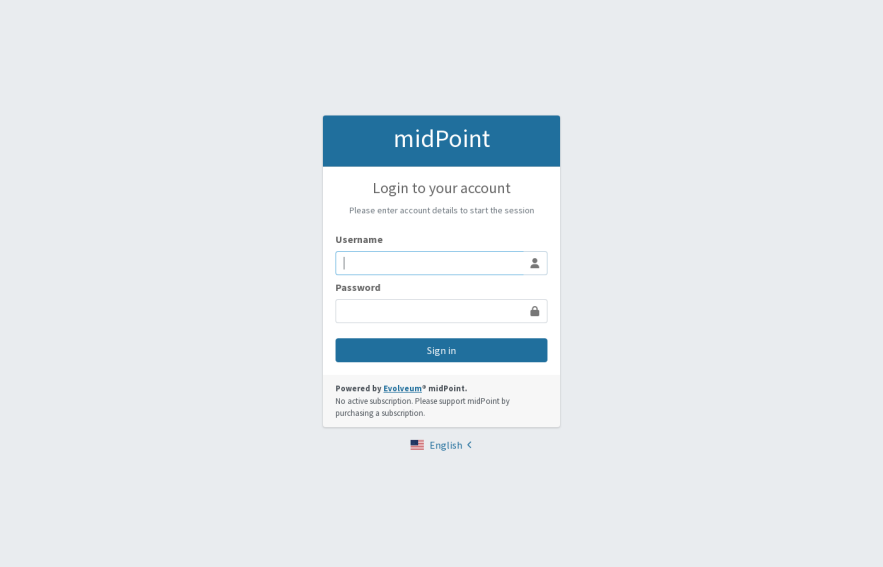


Figure 4: MidPoint GUI login (https://midpoint.$DOMAIN/midpoint/login).

Roles and users. Application roles created here are picked up by invite_bridge and synced to Invite. After a successful sync the Role's costCenter is invite:{id}. When Invite echoes a SCIM Group, the Role also stores extension/scimResource. Invite-provisioned guests appear under **Users** after accept (SCIM POST /Users); memberships are MidPoint role assignments. See also Figures 5 and 6 below.

5 Invite user from MidPoint

MidPoint administrators can start an Invite invitation without switching to the Invite application. The invite_bridge service:

1. Serves a small HTML form at https://invite-bridge.$DOMAIN/invite (HTTP Basic auth in this demo).
2. On startup, registers a MidPoint adminGuiConfiguration/additionalMenuLink labelled **Invite user** that points at that URL.
3. On submit, calls Invite POST /api/external/v1/invitations (email + Invite role id). **No MidPoint User is created** until the guest accepts and Invite provisions via SCIM.

Role eligibility. The form lists only MidPoint Roles that Invite has already SCIM-provisioned (they carry extension/scimResource, or legacy SCIM JSON in description) and that have an Invite role id (costCenter=invite:{id}). Roles that exist only in MidPoint, or only in Invite without a SCIM echo, are omitted.

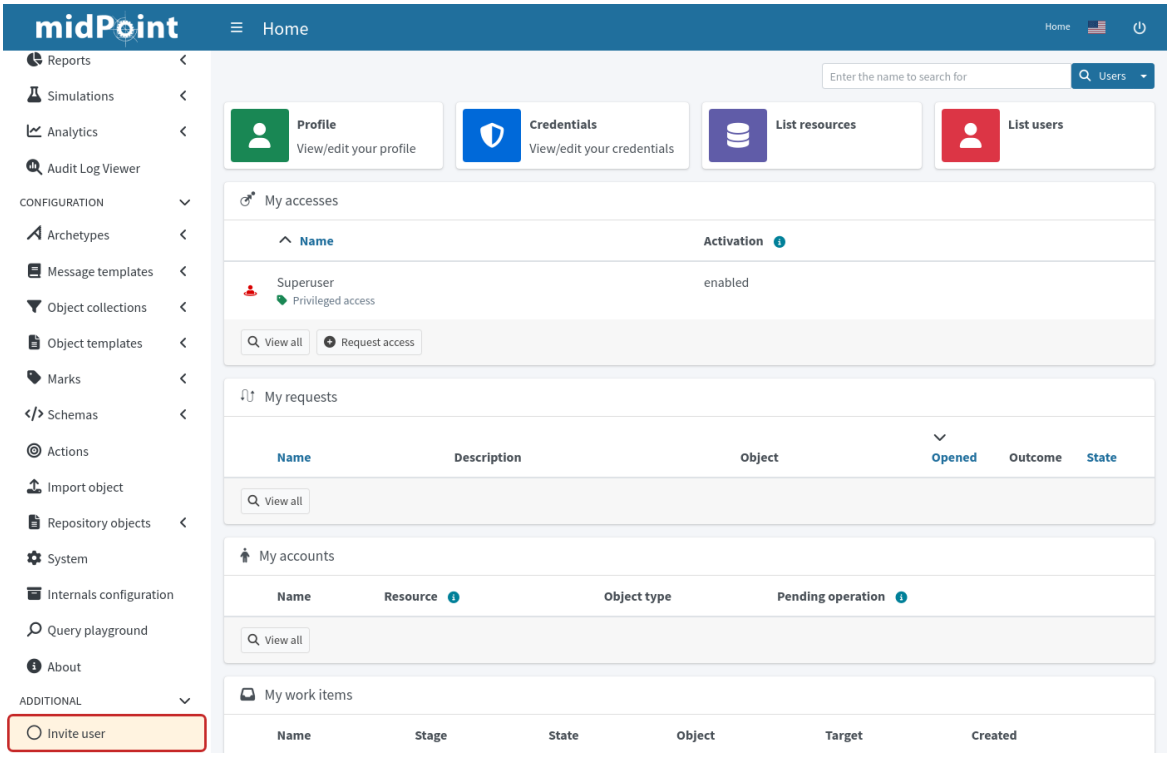


Figure 5: MidPoint GUI: **Invite user** under the ADDITIONAL menu (registered by invite_bridge as additionalMenuLink).

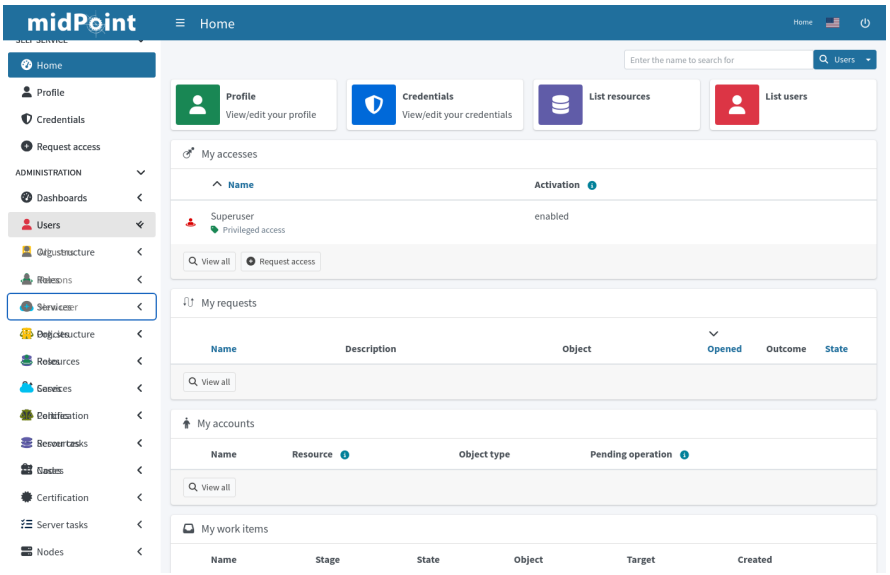


Figure 6: For comparison: MidPoint **Users** menu still offers **New user** (creates a MidPoint UserType immediately). **Invite user** is a separate path that only creates an Invite invitation.

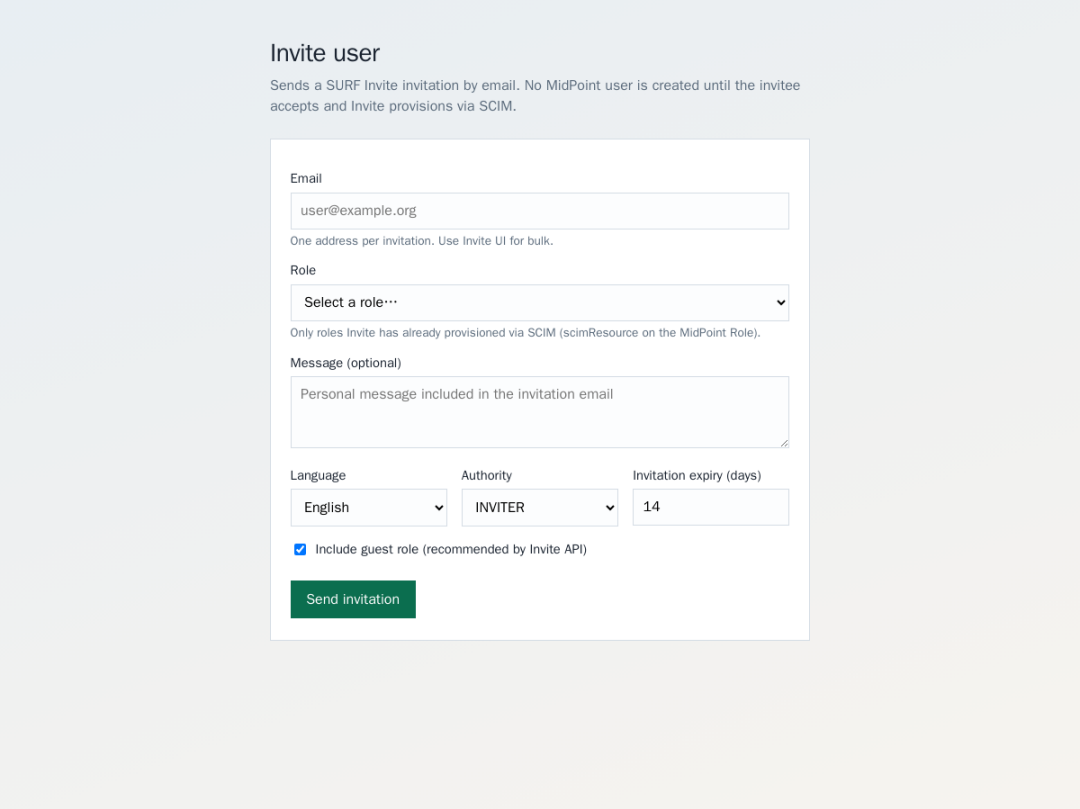


Figure 7: Invite user form served by invite_bridge: email, SCIM-eligible role, optional message, language, authority, and expiry. Submitting calls Invite POST /invitations.

6 OIDC application (membership pull)

The demo application at app.$DOMAIN is intentionally a pull authorization client. It is useful to verify what MidPoint/SCIM currently know about a user.

6.1 Operator flow

1. **Select lookup method:** MidPoint or SCIM (LDAP remains in the app code for parked stacks).
2. **Select required groups** listed by that backend.
3. **Login with OIDC** (SURFconext). After callback, resolve the user key from USER_LOOKUP_CLAIMS and fetch memberships with the same backend. Access is granted when selected ∩ resolved is non-empty.

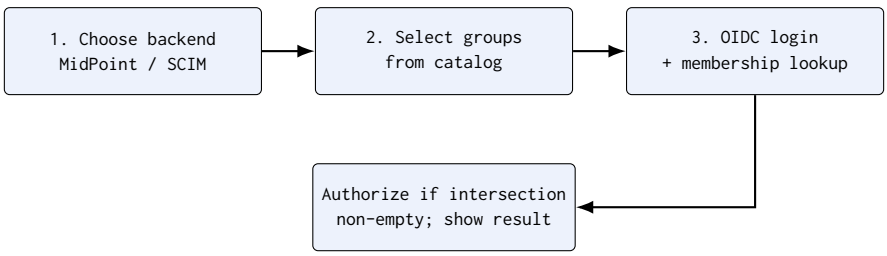


Figure 8: Application authorization wizard.

6.2 Screenshots

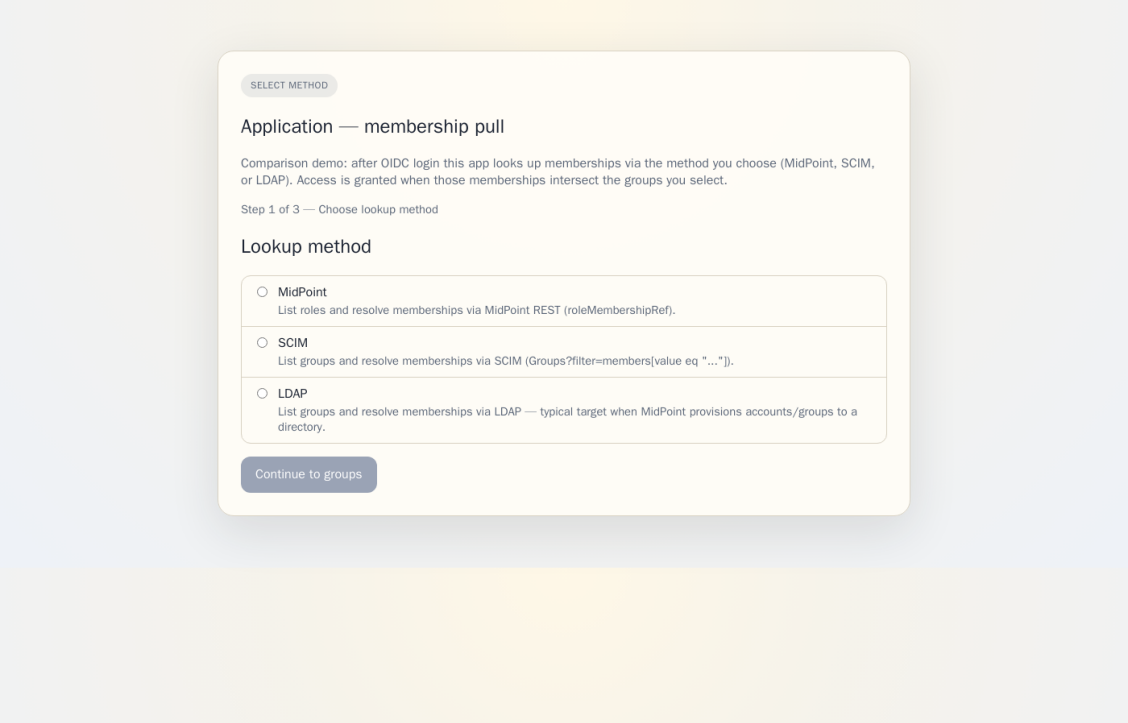


Figure 9: Application — Step 1: choose membership lookup backend.

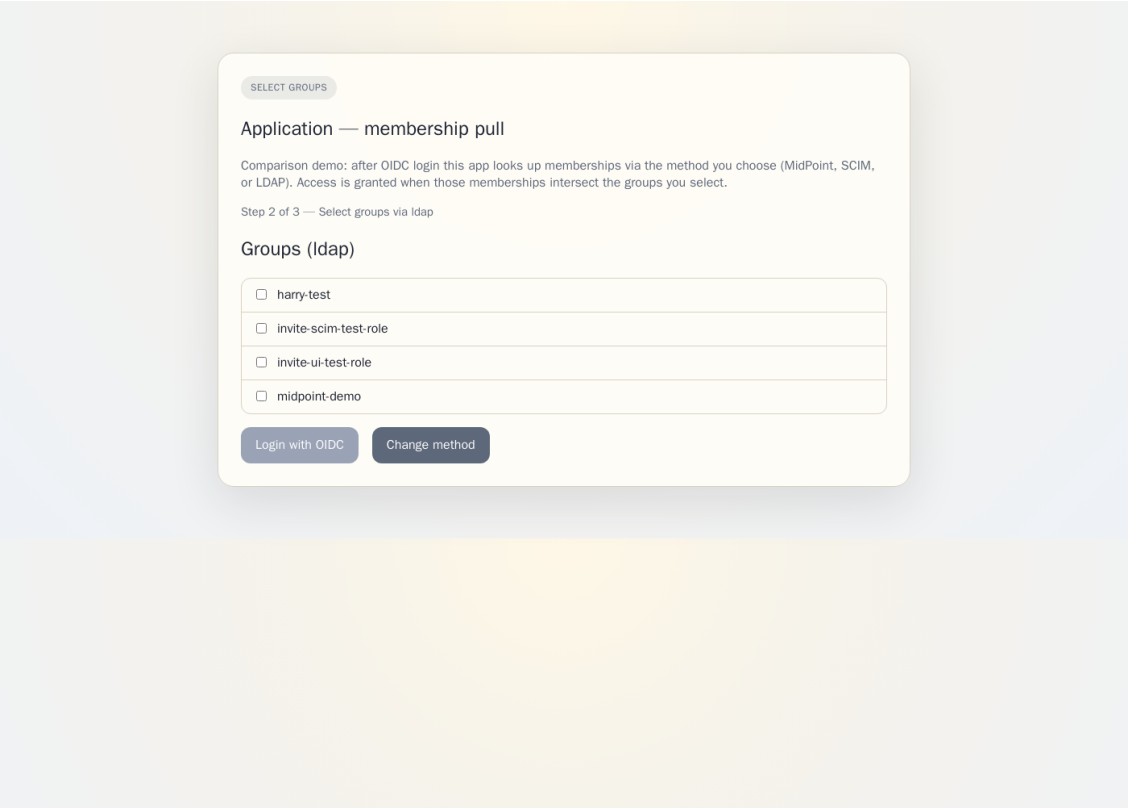


Figure 10: Application — Step 2: groups discovered from the selected backend.

6.3 Health endpoint

GET /health reports backend readiness. Example from the running pilot:


```
{
    "lookup_backends": {
        "ldap": {
            "error": null,
            "ready": true
        },
        "midpoint": {
            "error": null,
            "ready": true
        },
        "scim": {
            "error": null,
            "ready": true
        }
    },
    "oidc_claims": null,
    "oidc_ready": true,
    "oidc_scope": "openid profile email",
    "status": "ok",
    "user_lookup_claims": [
        "eduperson_principal_name"
    ]
}
```


7 Deployment and configuration

7.1 Start the stack

```
# with Traefik (default Makefile target)
make build
# or
docker compose -f compose.yml -f compose.traefik.yml up -d --build

# Toggle Invite/OIDC profile via ENV_FILE=.env.invite | .test.env in .env
# then recreate invite_bridge (+ application if OIDC differs)
```

7.2 Important environment variables

Variable	Purpose
DOMAIN	Base DNS name for Traefik hosts
ENV_FILE	Invite/OIDC profile (.env.invite or .test.env)
MIDPOINT_ADMIN_INITIAL_PASSWORD	MidPoint administrator password
INVITE_*	Invite external API + Manage application linkage
INVITE_BRIDGE_SYNC_MEMBERSHIPS	MidPoint unassign → Invite notify (default true)
INVITE_BRIDGE_PUBLIC_URL	Target of MidPoint **Invite user** menu link
INVITE_BRIDGE_UI_PASSWORD	HTTP Basic password for the invite form
OIDC_* / USER_LOOKUP_CLAIMS	Demo app SURFconext RP (from profile)
SCIM_API_KEY	Bearer token for Invite → SCIM
MIDPOINT_LEADING_GROUPS	SCIM Group echo matches existing MidPoint Roles
SCIM_IDEMPOTENT_USER_CREATE	Re-POST User reuses existing (avoids 409)

Table 3: Key configuration knobs (see .env.example and profile files).

7.3 Traefik hostnames

Assuming DOMAIN=pilot1.sram.surf.nl:

- Application: https://app.pilot1.sram.surf.nl/
- MidPoint: https://midpoint.pilot1.sram.surf.nl/midpoint/
- SCIM: https://scim.pilot1.sram.surf.nl/
- Invite user form: https://invite-bridge.pilot1.sram.surf.nl/invite

Configure the SURF Invite SCIM provisioner as described in Section 3.1 (Figure 3), and configure INVITE_* for invite_bridge (see repository README.md). Invite must use remote Manage so application metadata and SCIM provisioners resolve correctly.

8 Operational notes and lessons learned

- **MidPoint owns Roles; Invite owns invitations.** invite_bridge syncs Role create/delete to Invite; SCIM carries Users and memberships back into MidPoint. MidPoint **Invite user** only creates Invite invitations (roles must already have scimResource).
- **Drift prevention on MidPoint unassign.** Invite-SCIM memberships removed in MidPoint are notified to Invite via by_provisioned_scim. Local MidPoint revoke always wins if Invite notify fails.
- **Invite Manage must be remote.** If /api/external/v1/manage/providers only lists seed apps, Invite is on LocalManage—Roles show as “unknown in Manage” and SCIM may not fire.
- **INVITE_APPLICATION_MANAGE_TYPE** must be uppercase (SAML20_SP, OIDC10_RP, . . .).
- **MIDPOINT_LEADING_GROUPS.** Prevents Invite Group echoes from creating duplicate MidPoint Roles or deleting Roles on Group DELETE.
- **SCIM_IDEMPOTENT_USER_CREATE.** Invite may re-POST an existing guest; reuse avoids 409 and broken accept flows.
- **Lookup key alignment.** Application USER_LOOKUP_CLAIMS must match the MidPoint user name written by SCIM (often the eduID EPPN).

9 Parked components (not in default compose)

The following remain in the repository for experiments but are **not** started by compose.yml:

- OpenLDAP / phpLDAPadmin / MidPoint OpenLDAP resource and metarole-ldap-group post-initial objects (ldap/, midpoint/post-initial-objects/100-* / 110-*).
- Moodle / moodle_bridge / ConnId Moodle resource.
- OneRoster / NRPS LMS path.

  See repository TODO.md.

10 Summary

The stack demonstrates a MidPoint-leading Invite path:

$$\textbf{MidPoint} \xrightarrow{\textbf{invite_bridge}} \textbf{Invite} \xrightarrow{\text{SCIM}} \textbf{MidPoint}$$

(+ MidPoint membership revoke → Invite by_provisioned_scim; optional OIDC app membership pull)

Roles created in MidPoint appear in Invite for invitation workflows; administrators can start invitations from MidPoint's **Invite user** menu or from Invite itself; accepted guests are provisioned as MidPoint users with role assignments via SCIM; MidPoint unassigns of Invite-SCIM members notify Invite to prevent drift. Reusable custom pieces for other MidPoint installations are invite_bridge and the SCIM MidPoint plugin leading-mode behaviour.

A Quick reference commands

```
# Follow MidPoint -> Invite sync / revoke notify
docker compose -f compose.yml -f compose.traefik.yml logs -f invite_bridge

# Follow SCIM (Users/memberships and Group echoes)
docker compose -f compose.yml -f compose.traefik.yml logs -f scim

# Unit tests
cd invite_bridge && python -m unittest test_membership_sync.py -v

# Manual Invite role create
./scripts/invite-create-role.sh my-role-name

# Application health
curl -s https://app.$DOMAIN/health | jq .
```